Source Data: employees.csv

This is the exact dataset every example, output, and type check in this guide was run against (21 rows — includes a duplicate David row and missing salary/performance_score values, used intentionally to demonstrate real-world data cleaning).

employee_id	name	department	join_date	salary	performance_score	remote
1001	Alice	Engineering	2019-01-15	72000.0	4.2	True
1002	Bob	Sales	2019-03-29	65000.0	3.5	False
1003	Charlie	Engineering	2019-06-10	81000.0	—	True
1004	David	HR	2019-08-22	58000.0	3.9	False
1005	Eva	Sales	2019-11-03	—	4.0	True
1006	Frank	Engineering	2020-01-15	95000.0	4.5	True
1007	Grace	HR	2020-03-28	61000.0	3.2	False
1008	Heidi	Sales	2020-06-09	67000.0	3.8	True
1009	Ivan	Engineering	2020-08-21	88000.0	4.1	True
1010	Judy	HR	2020-11-02	59000.0	3.6	False
1011	Karl	Sales	2021-01-14	70000.0	3.9	True
1012	Liam	Engineering	2021-03-28	92000.0	4.7	True
1013	Mona	HR	2021-06-09	60000.0	3.4	False
1014	Nina	Sales	2021-08-21	66000.0	3.7	True
1015	Oscar	Engineering	2021-11-02	85000.0	4.3	True
1016	Priya	HR	2022-01-14	62000.0	3.5	False
1017	Quinn	Sales	2022-03-28	71000.0	3.9	True
1018	Rita	Engineering	2022-06-09	99000.0	4.8	True
1019	Sam	HR	2022-08-21	—	3.3	False
1020	Tara	Sales	2022-11-02	68000.0	3.6	True
1004	David	HR	2019-08-22	58000.0	3.9	False

Pandas Output Datatypes — Reference Guide (with function descriptions)

Every line below was actually executed against `employees.csv` and the type was captured with `type()`. Each function now has **what it does** in plain English, plus **what type it returns**, so you're never guessing from the name alone.

```
import pandas as pd
df = pd.read_csv("employees.csv", parse_dates=["join_date"])
```

---

1. Single column / single cell selection

```
salary_series = df['salary']
# WHAT IT DOES: grabs one column by name
# RETURNS: pandas.Series

salary_series_loc = df.loc[:, 'salary']
# WHAT IT DOES: same as above, written using label-based indexing (all rows, one named column)
# RETURNS: pandas.Series

single_value = df.loc[0, 'salary']
# WHAT IT DOES: gets the exact cell at row-label 0, column 'salary'
# RETURNS: numpy.float64 (a single number, not a Series)

single_row_series = df.loc[0]
# WHAT IT DOES: gets the entire row labeled 0, across all columns
# RETURNS: pandas.Series (index = column names, values = that row's data)

single_row_iloc = df.iloc[0]
# WHAT IT DOES: same as above, but by position (0 = first row) instead of by label
# RETURNS: pandas.Series
```

2. Multi-column / multi-row selection

```
two_cols_df = df[['name', 'salary']]
# WHAT IT DOES: selects two named columns at once (list of column names inside the brackets)
# RETURNS: pandas.DataFrame

loc_slice_df = df.loc[0:3, ['name', 'salary']]
# WHAT IT DOES: selects rows labeled 0 through 3 (inclusive), for the 'name' and 'salary' columns
# RETURNS: pandas.DataFrame

iloc_slice_df = df.iloc[0:3, 0:2]
# WHAT IT DOES: selects rows at positions 0,1,2 and columns at positions 0,1 (both exclusive of the end number)
# RETURNS: pandas.DataFrame
```

3. Boolean filtering & masks

```
mask_series = df['salary'] > 80000
# WHAT IT DOES: compares every value in the column to 80000, producing True/False for each row
# RETURNS: pandas.Series (dtype: bool) — this is the "mask" itself, not the filtered data

filtered_df = df[mask_series]
# WHAT IT DOES: keeps only the rows where the mask is True
# RETURNS: pandas.DataFrame

query_result_df = df.query("salary > 80000")
# WHAT IT DOES: same filtering as above, but written as a readable string expression instead of a boolean mask
# RETURNS: pandas.DataFrame
```

4. Missing data

```
null_mask_df = df.isna()
# WHAT IT DOES: checks EVERY cell in the dataframe and marks it True if it's missing (NaN/None), False otherwise
# RETURNS: pandas.DataFrame (same shape as df, but all boolean values)

null_count_series = df.isna().sum()
# WHAT IT DOES: adds up the True values (missing cells) column by column — because True counts as 1 in sum()
# RETURNS: pandas.Series (one number per column: how many nulls that column has)

total_nulls_int = df.isna().sum().sum()
# WHAT IT DOES: adds up the per-column counts above into one grand total for the whole dataframe
# RETURNS: numpy.int64 (a single number)

dropped_na_df = df.dropna()
# WHAT IT DOES: removes any ROW that has at least one missing value, anywhere in that row
# RETURNS: pandas.DataFrame (fewer rows than the original)

filled_series = df['salary'].fillna(0)
# WHAT IT DOES: replaces every missing value in the column with 0 (or mean/median/any value you pass in)
# RETURNS: pandas.Series
```

5. Duplicates

```
dup_mask_series = df.duplicated()
# WHAT IT DOES: flags each row True if it's an exact repeat of an earlier row, False if it's the first occurrence
# RETURNS: pandas.Series (dtype: bool)

dup_count_int = df.duplicated().sum()
# WHAT IT DOES: counts how many rows are duplicates (True values)
# RETURNS: numpy.int64

deduped_df = df.drop_duplicates()
# WHAT IT DOES: removes duplicate rows, keeping only the first occurrence of each
# RETURNS: pandas.DataFrame
```

6. Renaming / type casting

```
renamed_df = df.rename(columns={'name': 'n'})
# WHAT IT DOES: renames one or more columns using a dictionary of {old_name: new_name}
# RETURNS: pandas.DataFrame (original df is untouched unless you pass inplace=True)

cast_df = df.astype({'employee_id': 'int32'})
# WHAT IT DOES: forces a column's data type to change (e.g. int64 -> int32 to save memory)
# RETURNS: pandas.DataFrame

cast_series = df['department'].astype('category')
# WHAT IT DOES: converts a text column into pandas' "category" type — stores each unique value once
#               internally instead of repeating the string, which saves a lot of memory on repetitive text
# RETURNS: pandas.Series (dtype: category)
```

7. Feature engineering (apply / map / cut / dummies)

```
applied_series = df['salary'].apply(lambda x: x)
# WHAT IT DOES: runs your custom function on every value in the column, one at a time
# RETURNS: pandas.Series

mapped_series = df['remote'].map({True: 1, False: 0})
# WHAT IT DOES: replaces each value using a lookup dictionary (fast, simple recoding — no custom logic needed)
# RETURNS: pandas.Series

binned_series = pd.cut(df['salary'], 3)
# WHAT IT DOES: splits a continuous numeric column into a fixed number of equal-width buckets/bins
# RETURNS: pandas.Series (dtype: category — each value becomes a bin label like "(58000, 71667]")

dummies_df = pd.get_dummies(df['department'])
# WHAT IT DOES: one-hot encodes a categorical column — turns each unique value into its own True/False column
# RETURNS: pandas.DataFrame
```

8. GroupBy & aggregation

```
grouped_obj = df.groupby('department')
# WHAT IT DOES: splits the dataframe into groups by department, but does NOT compute anything yet —
#               it's a "lazy" object waiting for you to tell it what aggregation to run
# RETURNS: DataFrameGroupBy object (not a DataFrame or Series until you call something like .mean() on it)

group_mean_series = df.groupby('department')['salary'].mean()
# WHAT IT DOES: groups rows by department, then computes the average salary within each group
# RETURNS: pandas.Series (index = department names, values = average salary)

group_agg_df = df.groupby('department').agg(avg=('salary', 'mean'))
# WHAT IT DOES: same grouping, but lets you compute multiple named aggregations at once
# RETURNS: pandas.DataFrame

value_counts_series = df['department'].value_counts()
# WHAT IT DOES: counts how many times each unique value appears in the column, sorted by frequency
# RETURNS: pandas.Series (index = unique values, values = counts)

crosstab_df = pd.crosstab(df['department'], df['remote'])
# WHAT IT DOES: builds a frequency table cross-tabulating two categorical columns against each other
# RETURNS: pandas.DataFrame
```

9. Pivot / reshape / index operations

```
pivot_df = df.pivot_table(index='department', values='salary')
# WHAT IT DOES: summarizes data into a spreadsheet-style pivot table (default aggregation is mean)
# RETURNS: pandas.DataFrame

melted_df = df.melt(id_vars=['name'])
# WHAT IT DOES: converts columns into rows — turns a "wide" table into a "long" one
#               (each row becomes name/variable/value instead of one column per variable)
# RETURNS: pandas.DataFrame

indexed_df = df.set_index('employee_id')
# WHAT IT DOES: makes an existing column become the row index, instead of the default 0,1,2,3...
# RETURNS: pandas.DataFrame

reset_df = df.reset_index()
# WHAT IT DOES: undoes set_index — turns the current index back into a regular column,
#               and restores the default 0,1,2,3... index
# RETURNS: pandas.DataFrame
```

10. Sorting

```
sorted_df = df.sort_values('salary')
# WHAT IT DOES: reorders all rows based on the values in one (or more) columns
# RETURNS: pandas.DataFrame

rank_series = df['salary'].rank()
# WHAT IT DOES: assigns a rank number (1st, 2nd, 3rd...) to each value based on its size
# RETURNS: pandas.Series (dtype: float64, even though ranks are whole numbers — ties are averaged)

top3_df = df.nlargest(3, 'salary')
# WHAT IT DOES: returns the 3 rows with the highest values in the given column (faster than sort+head)
# RETURNS: pandas.DataFrame
```

11. Merge / concat

```
merged_df = df.merge(dept_info, on='department')
# WHAT IT DOES: joins two dataframes together based on matching values in a shared column (like a SQL JOIN)
# RETURNS: pandas.DataFrame

concat_df = pd.concat([df, df])
# WHAT IT DOES: stacks multiple dataframes on top of each other (or side by side with axis=1)
# RETURNS: pandas.DataFrame
```

12. Dates

```
year_series = df['join_date'].dt.year
# WHAT IT DOES: pulls just the year out of a datetime column, using the special .dt accessor
# RETURNS: pandas.Series (dtype: int32)

resampled_series = df.set_index('join_date').resample('YE')['salary'].mean()
# WHAT IT DOES: groups time-series data into yearly buckets and computes the average salary per year
# RETURNS: pandas.Series
```

13. Text (`.str`)

```
lower_series = df['name'].str.lower()
# WHAT IT DOES: lowercases every string in the column, using the special .str accessor for text columns
# RETURNS: pandas.Series

contains_series = df['name'].str.contains('a')
# WHAT IT DOES: checks if each string contains a given substring, returns True/False per row
# RETURNS: pandas.Series (dtype: bool)

len_series = df['name'].str.len()
# WHAT IT DOES: counts the number of characters in each string
# RETURNS: pandas.Series (dtype: int64)
```

14. Stats & correlation

```
corr_df = df[['salary', 'performance_score']].corr()
# WHAT IT DOES: computes how strongly each pair of numeric columns moves together (-1 to +1)
# RETURNS: pandas.DataFrame (a correlation matrix)

mean_float = df['salary'].mean()
# WHAT IT DOES: computes the average of a column
# RETURNS: numpy.float64

sum_float = df['salary'].sum()
# WHAT IT DOES: adds up all values in a column
# RETURNS: numpy.float64

max_float = df['salary'].max()
# WHAT IT DOES: finds the largest value in a column
# RETURNS: numpy.float64

nunique_int = df['department'].nunique()
# WHAT IT DOES: counts how many DISTINCT values exist in a column (duplicates don't count twice)
# RETURNS: plain Python int
```

15. Structural attributes (no parentheses — these are properties, not functions you call)

```
shape_tuple = df.shape
# WHAT IT DOES: tells you the dataframe's dimensions
# RETURNS: plain Python tuple -> (num_rows, num_columns)

columns_index = df.columns
# WHAT IT DOES: gives you the list of column names
# RETURNS: pandas.Index

dtypes_series = df.dtypes
# WHAT IT DOES: shows the data type of every column at once
# RETURNS: pandas.Series (index = column names, values = dtype of each)

index_obj = df.index
# WHAT IT DOES: gives you the row labels (the "index") of the dataframe
# RETURNS: pandas.RangeIndex by default, or pandas.Index after set_index()
```

16. Converting OUT of pandas into plain Python / NumPy

```
values_ndarray = df.values
# WHAT IT DOES: converts the entire dataframe into a raw 2D array, dropping column names and index
# RETURNS: numpy.ndarray (2D)

col_ndarray = df['salary'].values
# WHAT IT DOES: converts one column into a raw array, dropping the pandas index
# RETURNS: numpy.ndarray (1D)

col_ndarray2 = df['salary'].to_numpy()
# WHAT IT DOES: same as .values above — this is the modern, preferred way to do it
# RETURNS: numpy.ndarray (1D)

salary_list = df['salary'].tolist()
# WHAT IT DOES: converts a column into an actual plain Python list — no pandas or numpy involved anymore
# RETURNS: plain Python list

records_dict = df.to_dict()
# WHAT IT DOES: converts the whole dataframe into a dictionary (default shape: {column: {row_index: value}})
# RETURNS: plain Python dict

records_list_dict = df.to_dict(orient='records')
# WHAT IT DOES: converts the dataframe into a list where each row becomes its own dictionary —
#               this is the format most APIs/JSON payloads expect
# RETURNS: plain Python list of dicts -> [{'name': 'Alice', 'salary': 72000.0, ...}, ...]

csv_string = df.to_csv()
# WHAT IT DOES: converts the dataframe into CSV-formatted TEXT, without writing any file to disk
# RETURNS: plain Python str

unique_array = df['name'].unique()
# WHAT IT DOES: lists each distinct value in a column exactly once, in order of first appearance
# RETURNS: numpy.ndarray (on most pandas setups) — but can be a pandas StringArray on newer
#          pandas versions using the string dtype backend; run type(...) locally to confirm on your version
```

The Type Hierarchy — how to think about it

```
DataFrame          -> 2D table                    e.g. df[['a','b']], df.groupby().agg(), merges, filters
Series             -> 1D labeled column            e.g. df['col'], df.loc[0], groupby().mean()
numpy scalar        -> single number (np.float64/np.int64)   e.g. df['col'].mean(), df.loc[0,'col']
tuple / Index / dict / list / str / int  -> plain Python, once you explicitly convert out
                                          e.g. .tolist(), .to_dict(), .to_numpy(), .shape
```

The pattern to remember: - **2 dimensions in the result →** `DataFrame` - **1 dimension →** `Series` - **0 dimensions (a single cell/aggregate) → a NumPy scalar** (`numpy.float64` , `numpy.int64` , etc. — *not* a plain Python `float` / `int` , which matters if you're doing strict type checks or JSON-serializing later) - **You only get plain Python types (** `list` **,** `dict` **,** `str` **,** `tuple` **) when you explicitly ask to leave pandas** — via `.tolist()` , `.to_dict()` , `.to_csv()` (no path arg), or attributes like `.shape`

A gotcha worth internalizing (comes up in real pipelines)

```
import json

count = df.isna().sum().sum()      # numpy.int64
json.dumps({"nulls": count})       # ✖ TypeError: Object of type int64 is not JSON serializable
json.dumps({"nulls": int(count)})  # ✔ cast to plain Python int first
```

This exact issue shows up constantly when you're logging metrics from a pandas pipeline into an API response, a config file, or a model-monitoring dashboard — numpy scalar types silently break JSON serialization until you explicitly cast them.